

Documentacion Handoff

Sistema App CAIPI

Arquitectura, modulos, schema, estados,
backlog y guia para clonar o continuar el sistema.

Fuente: DOCUMENTACION_HANDOFF_SISTEMA.md

AVISO: Ignorar docs viejos de Firebase/Firestore en el repo.

Backend real = Supabase. Ver tambien seccion Roles/Permisos en el informe.

DOCUMENTACIÓN HANDOFF - Sistema App CAIPI

Para qué sirve este archivo: copiarlo a otra IA (o a esta misma en un chat nuevo) y decir:

"Implementa / continúa un sistema escolar igual a este. Usa este documento como fuente de verdad."

>

Fecha de este snapshot: julio 2026

Repo: app-caipi · App Flutter escuela_caipi · package Android com.escuela.caipi

>

IMPORTANTE: hay muchos .md viejos en el repo que hablan de Firebase/Firestore. El sistema real actual es Supabase (Postgres + Auth + RLS + Realtime + Storage). Ignora guías Firebase salvo como historia.

1. Qué es el sistema (en una página)

App móvil de gestión escolar para un CAIPI / preescolar / primaria pequeña-mediana.

Quién la usa

Rol en BD (usuarios.rol) | Quién | Qué hace

directora | Dirección | Todo: alumnos, pagos, personal, reportes, bitácoras, chat, etc.

profesor_admin | Profesora con más poder | Similar a profesora + permisos ampliados

profesor | Profesora | Opera según permisos granulares asignados

padre | Padre/tutor | Ve hijos, pagos, bitácora, eventos, autorizados, QR, chat

Stack real

Capa | Tecnología

App | Flutter 3.x, Dart SDK >=3.0.0 <4.0.0

Estado | provider (AuthService ChangeNotifier)

Navegación | go_router con redirect por login/rol

Backend | Supabase: Auth email/password, Postgres, RLS, Realtime, Storage

UI | Material + google_fonts (Poppins), colores en lib/config/app_colors.dart

PDF | pdf + printing (recibos y reportes)

Excel | excel + share/save

QR | qr_flutter

Notificaciones | flutter_local_notifications (+ listeners Realtime)

WhatsApp | Twilio opcional (lib/services/whatsapp_service.dart) - no es el canal principal

Plataformas

- Android: operativa (APK release se ha generado).

- iOS: código Flutter multiplataforma; no publicada; ios: false en launcher icons; falta Apple Developer + App Store.

2. Cómo está organizado el código

```
lib/  
main.dart          # Init Supabase, notificaciones, AuthService, GoRouter  
config/  
  supabase_config.dart # URL + anon key (o --dart-define)  
  app_colors.dart  
  twilio_config.dart   # Opcional WhatsApp  
models/             # DTOs fromJson/toJson (uno por entidad)  
services/            # Acceso a datos y lógica  
  auth_service.dart  
  supabase_service.dart # CRUD principal alumnos/pagos/etc.  
  permisos_service.dart  
  chat_service.dart  
  storage_service.dart  
  notification_service.dart  
  app_realtime_notifications.dart  
  recibo_pago_pdf.dart  
  reportes_pdf_service.dart  
  pdf_branding.dart  
  exportacion_pagos_excel.dart  
  solicitud_recogida_service.dart  
  whatsapp_service.dart  
routes/app_router.dart # TODAS las rutas  
screens/  
  login_screen.dart  
  cambiar_contrasena_screen.dart  
directora/          # Pantallas staff/dirección  
padres/              # Pantallas familia
```

```
chat/      # Chat padre ↔ escuela
widgets/   # Drawer, cards, paneles recogida, etc.
utils/
```

Patrón típico de pantalla

1. AppDrawer + AppBar
2. Datos con FutureBuilder / StreamBuilder / carga en initState vía Supabase.instance.client o SupabaseService
3. Navegación con context.go(...) / context.push(...)
4. Feedback con SnackBar

Auth

- Login: Supabase Auth -> carga fila usuarios donde id = auth.users.id
- AuthService expone isDirectora, esPadre, esProfesor
- Router redirige / -> /login o /directora o /padre

3. Base de datos (Supabase / Postgres)

3.1 Cómo levantar / migrar

No hay un solo SQL “perfecto” al día de hoy; el proyecto creció por parches.

Orden conceptual recomendado para un clon nuevo:

1. SQL_MAESTRO_COMPLETO.sql o DATABASE_COMPLETA.sql (schema base + roles/permisos)
2. Parches de pagos/becas/planes: FIX_PLANES_PAGO_Y_CONFIG.sql, FIX_SISTEMA_BECAS.sql, FIX_ALTA_ALUMNOS_PAGOS_Y Borrado.sql
3. Eventos/incidentes: EVENTOS_E_INCIDENTES_CORREGIDO.sql
4. Bitácora gastos: ADD_BITACORA_GASTOS.sql (+ grupos si aplica)
5. Chat: ADD_CHAT_PADRES_ESCUELA.sql
6. Solicitudes recogida: ADD_SOLICITUDES_RECOGIDA.sql
7. Entrevista: FIX_AGREGAR_ENTREVISTA_PADRES.sql
8. QR temporal: FIX_SISTEMA_QR_TEMPORAL.sql
9. RLS / permisos directora: FIX_PERMISOS_DIRECTORA_COMPLETO.sql, FIX_RLS_* según necesidad
10. Datos demo: DATA_INICIAL_COMPLETA.sql / INSERT_GRADOS.sql
11. Limpieza operativa (cuidado): LIMPIAR_DATOS_OPERATIVOS.sql

Config app: lib/config/supabase_config.dart

Proyecto actual de ejemplo: qxldfqnuwpuctajcazf.supabase.co (cambiar en un clon).

3.2 Tablas principales (modelo mental)

```
auth.users
├── usuarios (id = auth uid, rol, nombre, email...)

grados
profesores (usuario_id -> usuarios, grado_id -> grados)
alumnos (grado_id, padre_id -> usuarios)
├── personas_autorizadas
├── pagos
│   └── abonos (folios / parciales)
├── calificaciones
├── bitacora_diaria
├── control_salidas
├── incidentes (tipo -> tipos_incidentes)
├── entrevistas_padres (o nombre similar en SQL entrevista)
└── qr_temporales / solicitudes_recogida

configuracion_costos / planes de pago (mensualidades, inscripción, becas)
anuncios
eventos
menu_maternal
clases_extracurriculares + participantes_clases (o participantes_clase)
bitacora_gastos
notificaciones
conversaciones + mensajes_chat (1 conversación por padre)
roles + permisos + roles_permisos + usuarios_permisos (o permisos_usuario)
```

Galería: tabla puede existir en SQL maestro; en la app se eliminó a propósito (rutas comentadas). No reimplementar salvo que el cliente lo pida.

3.3 Pagos - esquema ACTUAL (no el del maestro antiguo)

El código Flutter espera algo como:

Campo | Uso
 fecha_vencimiento | (antes fecha_limite)
 estatus | pendiente \ | parcial \ | pagado \ | vencido \ | cancelado
 monto_pagado | suma de abonos
 forma_pago, notas, anio_escolar, tipo_pago, recibido_por_nombre | meta
 tabla abonos | cada pago parcial con folio

Al crear alumno, SupabaseService genera pagos iniciales (inscripción/mensualidades según plan 10/12 y beca). Si falla la generación, debe hacer rollback del alumno (evitar duplicados).

Ver: FIX_ALTA_ALUMNOS_PAGOS_Y Borrado.sql + lib/services/supabase_service.dart.

3.4 Seguridad

- RLS en casi todas las tablas: padres solo ven lo de sus hijos; staff según rol/permisos.
- Función típica: usuario_tiene_permiso(uuid, text).
- Directora suele bypass / permisos totales vía SQL de fix.
- Deletes críticos a veces usan .select() para detectar RLS que “borra 0 filas” en silencio.

4. Módulos - qué hace cada uno y estado

Leyenda: [OK] listo · [!] parcial / frágil · [X] no / descartado · [>>] mejora deseada

4.1 Núcleo

Módulo | Qué hace | Estado | Archivos clave
 Login / sesión | Email+password Supabase; carga perfil | [OK] | login_screen, auth_service
 Roles | 4 roles; dashboards distintos | [OK] | models/rol.dart, usuario.dart, router
 Permisos | Códigos tipo ver_alumnos, ver_pagos...; menú dinámico | [OK] | permisos_service, app_drawer, SQL permisos
 Cambiar contraseña | Usuario cambia su password | [OK] | cambiar_contrasena_screen
 Drawer / menú | Secciones por rol + permisos; header logo CAIPI | [OK] | app_drawer.dart

4.2 Alumnos y estructura

Módulo | Qué hace | Estado | Notas
 Grados | CRUD grados, activo/inactivo | [OK] | grados_screen, crear_grado_screen
 Alumnos | Lista, filtros, crear, editar, eliminar | [OK] | Alta genera pagos; cuidado RLS al borrar
 Padres | Lista, crear, ver detalle e hijos | [OK] | Edición padre puede ser limitada
 Profesoras | Lista, crear/editar, asignar grado | [OK] |
 Permisos por profesora | UI para togglear permisos | [OK] | permisos_profesor_screen

4.3 Cobranza

Módulo | Qué hace | Estado | Notas
 Pagos (directora) | Lista, semáforo, filtros, Excel | [OK] |
 Acreditar / abonos | Parcial o total, folio, recibo PDF | [OK] | acreditar_pago_screen, recibo_pago_pdf
 Config costos / planes | Montos inscripción, mensualidad, becas | [OK] | configuracion_costos_screen
 Pagos (padre) | Ve pagos de su hijo | [OK] | pagos_padre_screen
 Recibo PDF branding | Logo CAIPI en PDF | [OK] | pdf_branding.dart

4.4 Entrevista a padres

Qué hace | Estado | Notas
 Lista por alumno -> formulario largo -> guardar avance / completar -> PDF | [OK] | Debe ir ligada a alumnoid; padre_usuario_id = padre del alumno, no la directora
 Rutas | /directora/entrevistas, /directora/entrevista/crear?alumnoid=, editar | [OK]

4.5 Comunicación

Módulo | Qué hace | Estado | Notas
 Chat padre ↔ escuela | 1 conversación por padre; lista escuela + chat padre | [OK] | SQL ADD_CHAT_PADRES_ESCUELA; Realtime
 Anuncios | CRUD; alcance todos o por grado | [OK] |
 Eventos | CRUD staff; padres ven filtrados | [OK] |
 Incidentes + tipos | Severidad 1-5; catálogo tipos | [OK] | Padres ven en detalle hijo
 Notificaciones locales | Push locales al app | [!] | No es FCM completo de producción
 WhatsApp Twilio | Servicio + pantalla test | [!] | Test quitado del menú; opcional

4.6 Operación diaria

Módulo | Qué hace | Estado | Notas

Bitácora diaria | Comió, baño, siesta, ánimo, obs.; padres la ven | [OK] |
 Bitácora gastos | Gastos escuela + alcance/grupos | [OK] |
 Control entrada/salida | Quién trajo/recogió, horas, fecha | [OK] |
 Personas autorizadas | CRUD por alumno (staff y padre) | [OK] |
 Solicitudes de recogida | Padre solicita; escuela atiende panel | [OK] | ADD_SOLICITUDES_RECOGIDA, widgets panel
 QR temporal | Padre genera código temporal | [!] | Generación padre OK; escáner escuela incompleto en fase actual
 Menú maternal | Menú del día comida | [OK] | Pantallas existen
 Clases extracurriculares | CRUD clases | [OK] / [!] | Inscripción participantes puede estar menos pulida
 Calificaciones | Por alumno, materia, periodo, promedio | [OK] | Más útil primaria que maternal
 Galería fotos | - | [X] | Descartada a propósito
 Reportes PDF | Bitácora, gastos, entradas/salidas por fechas | [OK] | Solo directora; reportes_pdf_screen

5. Rutas actuales (resumen)

Definidas en lib/routes/app_router.dart.

Comunes: /login, /cambiar-contrasena

Directora / staff (ejemplos):

/directora, /directora/alumnos, .../crear, .../editar/:id,
 /directora/entrevistas, /directora/entrevista/crear, .../editar/:id,
 /directora/reportes-pdf, /directora/pagos, /acreditar-pago/:pagold,
 /directora/configuracion-costos, /directora/profesores, .../permisos,
 /directora/padres, /directora/personas-autorizadas/:alumnold,
 /directora/anuncios, /directora/eventos, /directora/incidentes, /directora/tipos-incidentes,
 /directora/grados, /directora/bitacoras, /directora/bitacora-gastos,
 /directora/control-salidas, /directora/calificaciones, /directora/menu-maternal,
 /directora/clases-extracurriculares, /directora/chat, /directora/chat/:id

Padre:

/padre, /padre/hijo/:id, /padre/hijo/:id/pagos, /padre/hijo/:id/bitacora,
 /padre/hijo/:id/personas-autorizadas, /padre/eventos, /padre/chat, /padre/qv-temporal

Redirect: no logueado -> login; directora -> /directora; padre -> /padre.

Profesoras suelen entrar por flujos de directora/staff según cómo esté el perfil (mismo árbol de pantallas + permisos).

6. Flujos de negocio críticos (para clonar bien)

Alta de alumno

1. Formulario (datos + grado + padre + plan/becha).
2. Insert alumnos.
3. Generar filas pagos iniciales.
4. Si pagos fallan -> borrar alumno (anti-duplicados).
5. Columnas deben coincidir con schema nuevo (fecha_vencimiento, estatus).

Acreditar pago

1. Abrir pago -> registrar abono (monto, forma, quien recibe).
2. Actualizar monto_pagado + estatus (parcial/pagado).
3. Ofrecer PDF recibo (también fallback si pago viejo sin abono).

Entrevista

1. Elegir alumno en lista.
2. Formulario multiparte; completado false/true.
3. PDF al completar.
4. Vincular padre real del alumno.

Chat

1. Padre abre/crea su única conversaciones row.
2. Mensajes en mensajes_chat; trigger actualiza preview.
3. Escuela ve lista de conversaciones.

Recogida

1. Padre crea solicitud / usa autorizados / QR temporal.
2. Escuela ve panel de solicitudes y registra control_salidas.

7. Estado global del proyecto (honestidad)

Área | % orientativo | Comentario

Core auth + roles + menú | ~95% | Estable

Alumnos / grados / padres / profes | ~90% | Detalles UX menores

Pagos + abonos + PDF + Excel | ~90% | Depende de SQL aplicado en BD

Bitácoras / control / autorizados | ~85-90% |

Chat + recogida | ~80-85% | Chat horario cutoff (ej. 16:00) puede faltar

QR extremo a extremo (escanear en puerta) | ~50-60% | Padre genera; scanner escuela pendiente

Reportes PDF | ~85% | 3 reportes principales

Notificaciones push cloud (FCM) | ~40% | Local + realtime parcial

WhatsApp | ~30% | Opcional / prueba

iOS publicación | ~10% | Código sí; tienda no

Galería | 0% | Cancelada

Documentación repo | Desordenada | Muchos MD obsoletos (Firebase)

Producción Android: usable en escuela real si Supabase Pro + RLS correctos + datos cargados.

No asumir que todos los SQL del repo ya están aplicados en el proyecto Supabase actual: verificar schema vs código.

8. Qué le falta / backlog útil

Alta prioridad (para “sistema redondo”)

1. Escáner QR en puerta (directora/staff) + validación contra QR temporal / autorizados.
2. Auditoría: confirmar en BD real que todos los FIX SQL de pagos/RLS/chat/recogida/entrevista están aplicados.
3. Push remotas (FCM o OneSignal) para padres offline.
4. Soft-delete o “activo=false” consistente en alumnos/pagos (evitar errores “no existe”).
5. Tests de humo de los 5 flujos: login, alta alumno, acreditar, bitácora, chat.

Media

6. Corte horario chat escolar.
7. Edición completa de padres si falta.
8. Participantes de extracurriculares más claro.
9. Panel web admin (hoy todo es móvil).
10. iOS: certificados, TestFlight, App Store.

Baja / opcional

11. WhatsApp automático de cobranza.
12. Facturación CFDI.
13. Multi-escuela (hoy es single-tenant por proyecto Supabase).
14. Galería (solo si el cliente insiste).

9. Dependencias Flutter relevantes (pubspec.yaml)

- supabase_flutter, provider, go_router
- google_fonts, cached_network_image, shimmer
- intl, image_picker, file_picker, share_plus, path_provider, file_saver
- qr_flutter, flutter_local_notifications
- excel, pdf, printing
- http, uuid, shared_preferences

Versión app: 1.0.0+1.

10. Cómo clonar / implementar uno “parecido”

Opción A - Fork del mismo producto (misma escuela o white-label)

1. Copiar repo.
2. Nuevo proyecto Supabase.
3. Ejecutar SQL en el orden de la sección 3.1.
4. Cambiar SupabaseConfig (URL + anon key).
5. Cambiar logos en assets/images/, colores, applicationId.
6. Crear usuario directora en Auth + fila usuarios rol directora.
7. flutter pub get -> flutter run / build APK.

Opción B - Reescribir desde cero con otra IA

Pasar este archivo completo + pedir:

“Flutter + Supabase, multi-rol, módulos de la sección 4 con estados [OK] primero. Schema Postgres sección 3. Flujos sección 6. No uses Firebase. Omite galería. Pagos con abonos y estatus. Chat 1:1 padre-escuela.”

Priorizar MVP:

1. Auth + usuarios + roles
2. Grados + alumnos + padres
3. Pagos + abonos + recibo
4. Bitácora + control salida + autorizados
5. Anuncios/eventos
6. Chat
7. Reportes PDF
8. QR/recogida

Opción C - Continuar este repo

Abrir Cursor en app-caipi, pegar este doc, y pedir tareas concretas del backlog §8.

Usar CodeGraph MCP (codegraph_context, codegraph_search) para ubicar símbolos antes de editar.

11. Archivos SQL / docs que SÍ importan vs ruido

Priorizar

- SQL_MAESTRO_COMPLETO.sql / DATABASE_COMPLETA.sql
- FIX_ALTA_ALUMNOS_PAGOS_Y_BORRADO.sql
- ADD_CHAT_PADRES_ESCUELA.sql
- ADD_SOLICITUDES_RECOGIDA.sql
- FIX_AGREGAR_ENTREVISTA_PADRES.sql
- FIX_PERMISOS_DIRECTORA_COMPLETO.sql
- FIX_SISTEMA_QR_TEMPORAL.sql
- ADD_BITACORA_GASTOS.sql
- PROPUESTA_COMERCIAL_CAIPI.md (venta, no técnico)
- Este archivo (DOCUMENTACION_HANDOFF_SISTEMA.md)

Tratar como históricos / desactualizados

- Cualquier guía Firebase / Firestore (DIAGRAMA_BD.md antiguo, CONFIGURACION_FIREBASE.md, partes de QUE_SIGUE.md)
- MODULOS_PENDIENTES.md (dice 0% en módulos que ya existen)
- Varios EJECUTAR_AHORA_*.md puntuales ya aplicados o supersedidos

12. Prompt listo para pegar a otra IA

Eres un desarrollador Flutter + Supabase. Vas a implementar (o continuar) un sistema escolar multi-rol llamado CAIPI según el documento DOCUMENTACION_HANDOFF_SISTEMA.md.

Reglas:

- Backend SOLO Supabase (Auth + Postgres + RLS + Realtime). Nunca Firebase.
- Roles: directora, profesor_admin, profesor, padre.
- Pagos usan estatus + abonos + fecha_vencimiento (no el schema viejo pagado/fecha_limite).
- Chat: una conversación por padre.
- No implementes galería de fotos.
- Android primero; iOS después.
- Respeta la estructura lib/models, services, screens/directora|padres|chat, go_router, provider.
- Antes de inventar tablas, alinea con la sección 3 del documento.
- Empieza por el MVP de la sección 10 Opción B, en ese orden.
- Documenta cada módulo con estado [OK]/[!]/[X] como en la sección 4.

Primera tarea: [DESCRIBIR AQUÍ].

13. Checklist rápido “¿está vivo?”

- [] Login directora OK
- [] Lista grados no vacía / no error RLS
- [] Crear alumno genera pagos sin duplicar
- [] Acreditar pago -> abono + PDF

- [] Bitácora se ve en padre
- [] Chat envía mensaje ambos lados
- [] Control salida + solicitud recogida
- [] Reportes PDF abren/comparten
- [] Entrevista guarda y exporta PDF
- [] APK Android instala en tablet real (DNS a Supabase OK)

14. Auditoría Roles y Permisos (julio 2026) - NO afinado

Veredicto: el sistema existe y “se ve” en el menú, pero no está bien cerrado. No confiar en él como control de seguridad único.

Qué sí existe

- Roles en usuarios.rol: directora, profesor_admin, profesor, padre
- Catálogo permisos + roles_permisos + extras usuarios_permisos
- RPC usuario_tiene_permiso(p_usuario_id, p_codigo_permiso) usada por PermisosService
- AppDrawer muestra/oculta bloques según ver_alumnos, ver_pagos, etc.
- UI PermisosProfesorScreen para dar/quitar permisos extra a una profesora
- En la versión “corregida” del SQL, directora = siempre true

Huecos importantes

1. Router (app_router.dart): al login solo bifurca directora -> /directora y el resto -> /padre. Un profesor puede acabar en dashboard de padre.
2. Usuario.esProfesor solo acepta profesor, no profesor_admin.
3. Permiso = menú, no ruta: si alguien navega a /directora/pagos a mano, la pantalla no revalida el permiso.
4. SQL inconsistente entre archivos: unos usan permisos.codigo, otros permisos.clave (SQL_MAESTRO / FIX_FUNCION_PERMISOS vs SISTEMA_PERMISOS_CORREGIDO).
5. Fallback del drawer: si falla la carga de permisos, rellena casi todos los ver_* en true.
6. Granularidad incompleta: el menú mira sobre todo ver_*; crear/editar/borrar casi no se chequean en botones/pantallas.
7. Defensa real: RLS en Supabase. Si RLS está flojo, el menú no salva.

Cómo debería quedar (meta)

- Router: directora|profesor|profesor_admin -> /directora; padre -> /padre
- esProfesor / esStaff que incluyan ambos roles de profesora
- Guard por ruta o wrapper RequierePermiso('ver_pagos') en pantallas sensibles
- Un solo SQL canónico de permisos (codigo, no clave) + función alineada al Flutter
- Sin fallback “dar todo”; si falla RPC -> menú mínimo o error
- RLS alineado a los mismos códigos / roles

Prioridad de arreglo

Alta - antes de producción con varias profesoras. Con solo directora + padres el riesgo es menor, pero el diseño actual es engañoso.

Fin del handoff. Actualizar este archivo cuando se cierre un ítem del backlog §8 o cambie el schema de pagos/chat.

ANEXO - Auditoria Roles y Permisos (julio 2026)

VEREDICTO: el sistema de permisos **NO** esta bien afinado. Funciona a medias.

Que SI hay:

- Tablas roles / permisos / roles_permisos / usuarios_permisos
- RPC usuario_tiene_permiso + PermisosService en Flutter
- Menu (AppDrawer) oculta secciones segun ver_*
- Pantalla para otorgar permisos extra a profesoras
- Directora suele recibir TRUE en la funcion SQL (bypass)

Problemas graves:

- 1) Router: profesor logueado puede ser mandado a /padre.
- 2) esProfesor solo detecta rol=='profesor', ignora 'profesor_admin'.
- 3) Permisos SOLO esconden el menu: rutas /directora/* sin guard.
- 4) SQL conflictivos (columna codigo vs clave en permisos).
- 5) Si falla el RPC, drawer da casi todos los ver_* = true.
- 6) Crear/editar/borrar casi no revisan permiso en pantallas.
- 7) Seguridad real depende de RLS en Supabase, no del menu.

Recomendacion: afinar en sesion dedicada (router + guards + SQL unico).